Machine Translated by Google

PGDM v2 — Deterministic Card Dealing Protocol (Continuation)

---

1. Objective of the PGDM v2

PGDM v2 (Protocol for Game Deck Management v2) defines the system of generating, shuffling and distributing cards as a **completely deterministic, verifiable and reproducible process,** eliminating any dependence on opaque RNG or centralized entities.

The formal objective is:

Ensure that any poker hand can be reconstructed bit by bit by any auditor node from the same cryptographic inputs.

---

2. Fundamental principles

PGDM v2 is based on four invariant principles:

2.1 Absolute Determinism

Same inputs ÿ same deck always.

f(seed, state, commits_players) ÿ single deck

2.2 External Non-RNG

There is no hidden external random source.

- No hardware RNG
- No entropy server • No operator's private seed

2.3 Global state dependence

The deck depends on the state of the system:

Machine Translated by Google

- current hand •
  commit history
- previous block hash

2.4 Universal reproducibility

Any node can rebuild:

- card order • exact
  distribution
- Distribution history

---

3. Construction of the global seed

The seed is not a unique value, but a cryptographic composition:

```
Seed_global = H(
    State_previous_hash
    + Commit_Player_Set
    + Time_window_nonce
    + Node_consensus_hash
)
```

Where:

- H = cryptographic hash function (SHA-256 or equivalent) •
  Commit_Player_Set = all action commits prior to distribution • Node_consensus_hash =
  consensus result between host nodes

---

4. Deck Generation

4.1 Base deck

The standard deck is defined as follows:

- 52 cards
- no wildcards

Machine Translated by Google

Representation:

Deck_0 = [Aÿ, 2ÿ, ..., Kÿ]

4.2 Deterministic Transformation

The deck is transformed through cryptographic permutation:

Deck_t = Permute(Deck_0, Seed_global)

4.3 Permutation Algorithm

The algorithm operates as follows:

1. Expand seed into pseudo-deterministic stream 2.
Map bytes ÿ swap indices 3. Execute
deterministic Fisher–Yates

```
for i from n-1 to 1:
   j = H(seed || i) mod (i+1)
   swap(deck[i], deck[j])
```

---

5. Critical property of the system

In traditional systems:

The shuffle is unrepeatable.

In PGDM v2:

The shuffle is a fixed mathematical function.

---

6. Card Distribution

6.1 Distribution Order

Machine Translated by Google

The cards are assigned in sequential order from the permuted deck:

Player_k receives deck[position_k]

6.2 Consistency Rule

All nodes must derive:

- same deck
- same assignment • same order of action

---

7. Validation between nodes

Each auditor node executes:

7.1 Local playback

Deck_rebuilt = PGDM_v2(State_snapshot)

7.2 Comparison

```
if Deck_rebuilt == Deck_consensus:
   VALID
else:
   REJECT_STATE
```

---

8. Prevention of tampering

PGDM v2 eliminates classic vectors:

Attack vector	Mitigation
Dealer manipulation	Removed (no central RNG)
Seed bias	Seed depends on consensus
Replay fraud	Hash chain prevents modification

Machine Translated by Google

Hidden shuffle	Reproducible public permutation

---

9. Relationship with LCCP (Crack Chain Core)

PGDM v2 is not an isolated system:

- It runs within the **LCCP protocol hand cycle** • Each deck is a “verifiable sub-state”
- Each hand is a logical block within the chain of events

---

10. Emergent property of the protocol

PGDM v2 turns poker into:

A deterministic information distribution system under distributed consensus.

This implies:

- There is no “hidden luck” •
  Only dependence on initial state and human actions • The result is computationally derived, not generated

---

11. PGDM v2 Design Status

The protocol is defined as:

```
PGDM_v2 =
    { seed_generation,
    deterministic_shuffle,
    sequential_deal,
    cross_node_validation
}
```

ÿ Next logical block

Machine Translated by Google

The next natural technical level is:

“LCCP Hand Lifecycle Protocol”

LCCP Hand Lifecycle Protocol

(Crack Chain Poker—Execution Layer Specification)

---

1. Formal definition

The **LCCP Hand Lifecycle Protocol (HLCP)** defines the complete execution cycle of a poker hand within the Crack Chain network, from its creation to its final settlement in a distributed state.

It is modeled as a **deterministic distributed state machine,** executed on cross-validated host nodes.

---

2. General model

A hand is a finite process:

Hand = { S0 ÿ S1 ÿ S2 ÿ ··· ÿ Sf }

Where:

- S0 = table creation / initialization
- Sf = final state settlement (distributed pot + hash closure)

---

3. Hand cycle states

3.1 STATE 0—INITIALIZATION

Function:

Machine Translated by Google

Create a deterministic table instance.

Entrance:

• Player list (wallet-based IDs) • Active host nodes (n = 40 standard) • Table parameters (blinds, format, rules)

Exit:

Table_State_0 = Hash(config + node_set + time_window)

3.2 STATE 1 — SEED SYNCHRONIZATION

Function:

Synchronize system base entropy.

Process:

• Each node generates a partial seed contribution • A global seed consensus is reached

Seed_global = Consensus(Seed_node_1 ... Seed_node_n)

3.3 STATE 2 — DECK GENERATION (PGDM v2)

Function:

Generation of the deterministic deck.

• PGDM v2 is running • The deck is identical on all nodes

Exit:

Deck_t = PGDM_v2(Seed_global, State_0)

3.4 STATE 3 — DEALING PHASE

Function:

Machine Translated by Google

Sequential distribution of cards.

Rules:

• Deterministic round-robin distribution • Each action is a signed event

Player_i ÿ Card_j = Deck_t[position]

3.5 STATE 4 — ACTION LOOP (CORE GAME LOOP)

Function:

Execution of betting rounds.

Every action is an event:

```
Action_t =
    { player_id,
    type (fold/call/raise/check),
    amount,
    signature,
    timestamp_hash
}
```

Validation by node:

Each node checks:

• Valid signature • Consistent sequence • sufficient balance • transition allowed

---

State transition:

S_t+1 = Transition(S_t, Action_t)

3.6 STATE 5 — CONSENSUS VALIDATION LOOP

Machine Translated by Google

Function:

Synchronize state between host nodes.

Ruler:

Valid(S_t) = majority(hash(S_t)) >= 2/3 nodes

If it fails:

- Rollback to the last valid state • Divergent node isolation

---

3.7 STATE 6 — SHOWDOWN (IF APPLICABLE)

Function:

Final assessment of active hands.

Process:

- Deterministic reconstruction of combinations • Identical evaluation at all nodes

Winner = f(hands, Deck_t, ruleset)

3.8 STATE 7 — POT SETTLEMENT

Function:

Potential distribution.

Ruler:

- Deterministic calculation of the winner • Cross-validation of payout

Balance_update = deterministic_distribution(Winner)

3.9 STATE 8 — FINALIZATION

Machine Translated by Google

Function:

Closing the hand as a verifiable block.

Final output:

```
Hand_Block = Hash(
    S0 ÿ Sf,
    Deck_t,
    Action_log,
    Consensus_log
)
```

This block is added to the system ledger.

---

4. Properties of the HLCP cycle

4.1 Logical Immutability

Once the hand is finished:

- The history cannot be altered. •
  Any change breaks the hash chain.

4.2 Total reproducibility

Any node can rebuild:

- Full deck • Actions
- final result

4.3 Central server independence

It does not exist:

- Central dealer •
  Central RNG • Single
  resolution authority

Machine Translated by Google

---

5. Relationship with PGDM v2

HLCP depends directly on:

- PGDM v2 ÿ deck generation • Global seed ÿ initial entropy • Consensus layer ÿ state validation

Structural relationship:

HLCP = Execution_Layer(PGDM_v2 + LCCP_Consensus + State_Machine)

---

6. Emergent property of the system

The complete system behaves as follows:

A deterministic game execution network with distributed real-time verification.

This implies:

- Each hand is a “logical block” • Each action is a verifiable transition • The result is mathematically reconstructible

---

7. Final state of the protocol

HLCP is defined as:

```
HLCP =
  { initialization,
  seed_sync,
  pgdm_deck_generation,
  dealing,
  action_loop,
  consensus_validation,
  showdown,
  settlement,
  finalization
```

Machine Translated by Google

}

ÿ Next logical level

The next natural module of the Crack Chain system is:

“LCCP Node Host Incentive & Operational Model”

LCCP Node Host Incentive & Operational Model

(Crack Chain Poker — Economic + Operational Layer Specification)

---

1. Formal definition

The **LCCP Node Host Incentive & Operational Model (NHIO Model)** defines how **host nodes** support, operate, and are incentivized within the Crack Chain Poker network.

This model connects:

- Technical operation of the system •
  Economic sustainability of the network •
  Distribution of execution load • Stability of the
  distributed consensus

---

2. Basic principle of the model

The system is built under a structural constraint:

The network must be self-sustaining without depending on a central server or external subsidies.

Formally:

Network_Operational_Cost ÿ Network_Fees_Generated

Machine Translated by Google

3. Host node structure

A standard set is defined:

N = 40 Host Nodes (standard cluster)

Each node performs simultaneous functions:

3.1 Operational function

- Game state execution • Action validation
- participation in consensus

3.2 Infrastructure Function

- Hand processing • State replication
- PGDM v2 verification

3.3 Economic Function

- receives compensation proportional to workload

---

4. Operational load model

Each node has a load variable:

L_node = f(hands_processed, validation_ops, consensus_participation)

Where:

- Higher traffic ÿ higher load • Higher load ÿ higher proportional reward

---

5. Incentive Model

5.1 Economic unit of the system

Machine Translated by Google

The base unit is:

Fee per hand played (Hand Execution Fee)

5.2 Fee distribution

Each hand generates a total fee:

F_total = rake_per_hand + operational_fee

Distribution:

```
F_total ÿ {
   Host_nodes (70–85%),
   Network_reserve (10–20%),
   Audit_layer (5–10%)
}
```

---

6. Distribution between nodes

Each node receives proportionally:

Reward_i = F_host * (L_i / ÿ L_all_nodes)

Where:

- L_i = load of node i •
  dynamic distribution based on actual participation

---

7. Incentive for correct behavior

The model penalizes invalid behavior:

7.1 Penalty

- nodes with desynchronization • nodes
  that fail validation

Machine Translated by Google

• nodes with hash inconsistencies

Reward_i = Reward_i - penalty_factor

7.2 Exclusion from consensus

If a node diverges repeatedly:

• is removed from the active pool • loses participation in fee distribution

---

8. System operating model

The system functions as a **distributed execution network remunerated by actual activity.**

Operational workflow:

1. Player executes action 2. Node processes event 3. Consensus validates
4. System records transition. 5. Fee is generated. 6. Nodes receive proportional reward.

---

9. Economic balance of the system

Stability condition:

ÿ Rewards_nodes + Reserve + Audit_cost ÿ Total_fees_generated

This guarantees:

• Network sustainability • Absence of structural deficit
• independence from external subsidies

Machine Translated by Google

10. Structural incentive (key to design)

The model does not reward:

- arbitrary volume
- system exploitation

The model rewards:

- **Real participation in consensus • State processing • Correct validation of PGDM v2 outputs**

---

11. Emergent property of the system

The NHIO Model transforms the network into:

a distributed computing infrastructure system paid for per execution of verifiable events

This implies:

- The nodes are not passive servers. • They are **economic actors in the execution of the game .** • The network is simultaneously: • a game engine • a settlement system • a compensation system

---

12. Relationship with previous layers

12.1 HLCP (Hand Lifecycle Protocol)

ÿ generates economic events (hands)

12.2 PGDM v2

Machine Translated by Google

ÿ ensures input integrity (cards)

12.3 LCCP Consensus Layer

ÿ validates state

12.4 NHIO Model (this document)

ÿ distributes economic value

---

13. Final definition of the model

```
NHIO =
    { load_measurement,
    fee_generation,
    proportional_distribution,
    penalty_system,
    node_economic_balance
}
```

ÿ Closure of the conceptual system

With this module, the Crack Chain system is now complete in 4 layers:

1. PGDM v2 ÿ deterministic randomness
2. HLCP ÿ Hand execution 3. LCCP Consensus ÿ Distributed validation 4. NHIO Model ÿ Node economy

---

“Formal System Architecture Specification (Crack Chain Core Protocol)”

Formal System Architecture Specification (Crack Chain Core Protocol)

(Core Integration Layer—System-Wide Technical Specification)

Machine Translated by Google

1. Formal definition

The **Crack Chain Core Protocol (CCCP)** is the system architecture layer that integrates all previous modules into a single coherent operational structure.

Its function is to unify:

• **PGDM v2** ÿ deterministic deck generation • **HLCP** ÿ complete hand cycle • **LCCP Consensus Layer** ÿ distributed state validation • **NHIO Model** ÿ incentives and economic operation of nodes

The result is a poker system executed as a **verifiable distributed protocol,** not as a centralized casino.

2. Objective of the protocol

The main objective of the CCCP is to define an architecture where:

1. Each game event is reproducible, 2. Each state transition is validatable, 3. Each node has an explicit function, 4. The entire system is auditable in real time, 5. Operation does not depend on a central authority.

3. System Layers

The protocol is divided into four functional layers:

```
CCCP = {
    Layer_1: Deck_Determinism,
    Layer_2: Hand_Execution,
    Layer_3: Distributed_Consensus,
    Layer_4: Node_Incentive_And_Settlement
}
```

Machine Translated by Google

3.1 Layer 1 — Deck Determinism

Responsible for:

- Construct the global seed • Derive the deck • Ensure reproducible distribution • Remove opaque RNG

This block is implemented by **PGDM v2.**

3.2 Layer 2 — Hand Execution

Responsible for:

- create the hand instance
- Execute player actions • Resolve bets • Reach the showdown or logical close

This block is implemented by **HLCP.**

3.3 Layer 3 — Distributed Consensus

Responsible for:

- Validate each state transition • Accept or reject events • Synchronize host nodes • Maintain ledger consistency

This block corresponds to the LCCP consensus layer.

3.4 Layer 4 — Node Incentive and Settlement

Responsible for:

- Measure operational workload • Distribute fees
- penalize divergent nodes • financially support the network

Machine Translated by Google

This block corresponds to the **NHIO Model.**

---

4. Architectural Principles

4.1 Determinism

Same input ÿ same result.

Input_t = Input_t' => Output_t = Output_t'

4.2 Reproducibility

Any authorized node can unambiguously reconstruct the complete state of a hand.

4.3 Distribution of control

There is no single master server.

Control is distributed among host nodes with cross-validation.

4.4 Logical Immutability

Once a state is accepted, any future alteration breaks the continuity of the hash chain.

---

5. System Components

5.1 Host Node

Operational node that:

- executes table logic

Machine Translated by Google

• validates partial state •
participates in consensus •
records events

5.2 Auditor Node

Verification node that:

• Reconstructs hands •
Compares hashes •
Detects divergences •
Generates consistency evidence

5.3 Player Node

Interaction node that:

• Sign actions • Send
game decisions • Receive updated
status

5.4 Settlement Engine

Engine that:

• liquidate pot •
update balances • close
hand
• outputs final result block

---

6. Communication Model

All internal communication of the protocol is based on signed and versioned messages.

6.1 Minimum message structure

Message = {

Machine Translated by Google

```
    message_type,
    protocol_version,
    hand_id,
    sender_id,
    payload,
    signature,
    previous_hash
}
```

6.2 Transmission Rules

• Each message must be signed. • Each
message must reference a previous state. • Every message
must be verifiable by external nodes. • Out-of-sequence messages are
rejected.

---

7. Ledger structure

The system maintains one logical ledger per hand and one aggregated ledger per table.

7.1 Hand Ledger

Contains:

• seed •
deck
• actions •
validations
• final result

7.2 Table Ledger

Contains:

• Hand history
• active nodes • load
status • integrity metrics

Machine Translated by Google

8. Global protocol flow

The complete system cycle is defined as follows:

Initialization ÿ
Seed Consensus
ÿ PGDM v2 Deck Generation ÿ Hand
Execution ÿ Action
Validation ÿ Consensus
Confirmation ÿ Settlement

ÿ Finalization ÿ
Ledger Commit

---

9. System state model

The system can be expressed as a hierarchical state machine.

```
System_State_t =
    { network_state,
    table_state,
    hand_state,
    node_state,
    settlement_state
}
```

Each substate must remain consistent with the others.

---

10. Coherence rules

A system is valid if it simultaneously fulfills:

```
Valid_System =
    Valid(Network_State) ÿ
    Valid(Table_State) ÿ
    Valid(Hand_State) ÿ
    Valid(Node_State) ÿ
    Valid(Settlement_State)
```

Machine Translated by Google

---

11. Rejection Rule

If there is an inconsistency at any of these levels:

- Invalid seed
- non-reproducible deck •
unsigned action •
inconsistent payout •
inconsistent hash

so:

- the state is rejected • the last
valid state is retained
- the divergent node is marked

---

12. Failure Management

12.1 Local failure

If a node fails locally:

- Its partial state is frozen •
Another node can reconstruct the transition
- The hand continues if the consensus remains valid

12.2 Consensus failure

If consensus does not reach a threshold:

- The hand is paused •
The last valid block is synchronized • The
validation round is re-evaluated

---

13. Validity threshold

The network adopts a logic of acceptance by qualified majority.

Machine Translated by Google

Accept(State_t) if Validations ÿ 2/3 of Active Host Nodes

This protects against:

- isolated nodes •
- manipulation attempts • unauthorized local forks

---

14. Economic integration

The protocol also defines how economic value enters the system and how it is distributed.

14.1 Value Input

Each hand generates an operating fee.

14.2 Distribution

That fee feeds:

- host nodes • network reservation
- audit layer • protocol sustainability

---

15. Emergent Property

Crack Chain Core Protocol does not act as a poker app.

It acts as:

a distributed operating system for verifiable poker games, with deterministic execution, continuous auditing, and automatic infrastructure compensation.

---

16. Final definition of the Core Protocol

CCCP = {

Machine Translated by Google

```
    deterministic_deck_layer,
    hand_lifecycle_layer,
    distributed_consensus_layer,
    node_incentive_layer,
    settlement_and_audit_layer
}
```

---

17. Technical Conclusion

With this specification, Crack Chain ceases to be a collection of loose ideas and becomes a **complete protocol stack:**

• cryptographic base • hand engine
• distributed consensus •
financial settlement • reproducible audit

In other words: a comprehensive protocol, not a traditional poker room.

ÿ Next natural block

The next logical step is:

“Protocol Versioning, Governance & Upgrade Rules”

This defines how the system evolves without breaking compatibility or structural integrity.

Protocol Versioning, Governance & Upgrade Rules

(Crack Chain Core Protocol—Evolution Control Layer)

---

1. Formal definition

The **Protocol Versioning, Governance & Upgrade Rules** define how Crack evolves Unbroken chain:

Machine Translated by Google

• node compatibility, • state
reproducibility, • ledger integrity,
• historical validity of
previous hands.

Its function is not political in an abstract sense, but **operational:** to control protocol changes with traceability, security and functional continuity.

---

2. Guiding principle

Every system change must comply with this rule:

New_Version must preserve:

historical_validity

deterministic_replay

consensus_integrity

node_compatibility_window

If an update breaks any of those points, it cannot go directly into production.

---

3. Versioning scheme

The protocol uses structured semantic versioning:

CCCP = MAJOR.MINOR.PATCH

3.1 MAJOR

Changes that affect:

• consensus logic, • table
rules, • deck structure,

Machine Translated by Google

• message format, • status
validation.

A MAJOR change may require coordinated node migration.

3.2 MINOR

Backward compatible changes that add:

• New optional rules, • Performance
improvements, • Audit optimizations,
• Non-destructive functional extensions.

3.3 PATCH

Minor corrections that do not alter:

• base rules, • state
format, • ledger compatibility,
• protocol determinism.

---

4. Version Compatibility

The system must maintain a **coexistence window** between active versions.

4.1 Coexistence Rule

Version_n and Version_n-1 may coexist

if replay_consistency == true

and consensus_rule_set is not broken

This allows:

• gradual node upgrade,

Machine Translated by Google

• cross-validation between versions, • orderly
migration without a complete break.

4.2 Rejection Rule

A version is rejected if:

• cannot reconstruct previous states, • introduces
ambiguity in PGDM v2, • breaks HLCP logic, •
alters the final settlement, •
modifies incompatible signatures
or hashes.

---

5. Governance model

Crack Chain's governance is not based on conventional political voting, but on **technical acceptance of change.**

5.1 Governing Unit

The decision unit is the **Protocol Change Proposal.**

CCP = Protocol Change Proposal

Each CCP must specify:

• objective of the change, •
technical scope, •
affected modules, • impact
on compatibility, • integrity testing, •
migration plan.

5.2 Government Actors

5.2.1 Host Nodes

Machine Translated by Google

• assess operational impact, •
validate compatibility, • perform
migration if necessary.

5.2.2 Auditor Nodes

• They verify that the change does not alter historical replay,
• they validate hashes and transitions,
• they certify the consistency of the upgrade.

5.2.3 Core Maintainers

• propose specification changes, • draft new
versions, • document transition rules.

---

6. Proposal cycle

An update follows this flow:

Draft ÿ Review ÿ Simulation ÿ Validation ÿ Activation ÿ Monitoring

6.1 Draft

The proposed change is written with formal specification.

6.2 Review

The host and auditor nodes examine the impact.

6.3 Simulation

The change is tested on a deterministic reproduction environment.

6.4 Validation

It is verified that:

• does not break consensus,
• does not alter historical positions,

Machine Translated by Google

- does not introduce ambiguity.

6.5 Activation

The version is activated only when there is a sufficient compatibility threshold.

6.6 Monitoring

Post-activation behavior is observed and compared with the expected model.

---

7. Activation Rule

A new version only becomes effective if it meets the following requirements:

Valid_Upgrade =

deterministic_replay_ok

ÿ consensus_ok

ÿ settlement_ok

ÿ node_compatibility_ok

If a condition fails, the system remains in the previous version.

---

8. Governance of critical changes

There are changes that cannot be made silently.

8.1 Critical Changes

They include:

- modification of the distribution algorithm, • change in deck rules, • alteration of the consensus threshold, • change in message format structure,

Machine Translated by Google

• change in settlement logic.

These changes require:

• explicit notification, • planned migration, • audited validation, • confirmation of active nodes.

8.2 Non-critical changes

They include:

• Internal refactoring, • Visual improvements, • Log optimization, • Improved error handling, • Non-functional performance improvements.

---

9. Version testing environment

Every upgrade must go through a deterministic validation environment.

9.1 Internal testnet requirements

• Recreate historical hands, • Simulate consensus between nodes, • Validate final settlement, • Compare hashes with reference.

9.2 Approval criteria

A version passes if:

• the replay produces the same result, • the consensus converges, • there is no state divergence, • the ledger remains consistent.

Machine Translated by Google

10. Rollback Rule

If a version generates inconsistency:

Rollback_to = last_valid_version

Rollback conditions:

• hash divergence, •
consensus failure, •
node incompatibility, • settlement
error, • PGDM v2 model
breakage.

The rollback must be reversible and auditable.

---

11. Change Log

Every update is recorded in a formal changelog.

11.1 Minimum fields

```
{
    version_id,
    timestamp,
    authoring_node,
    affected_modules,
    change_type,
    compatibility_status,
    audit_signature
}
```

This record is not decorative: it is part of the trust system.

Machine Translated by Google

---

12. Emergent property of the governance

The governance of the system does not seek broad political consensus, but **verifiable technical stability.**

That means the system prioritizes:

- integrity,
- continuity,
- traceability,
- compatibility,
- determinism.

It does not prioritize popularity of the change or speed of publication over structural safety.

---

13. Final definition of the module

```
Protocol_Versioning_Governance_Upgrade_Rules = {

    semver_scheme,

    compatibility_window,

    proposal_cycle,

    activation_rules,

    rollback_rules,

    changelog_registry

}
```

---

14. Technical Conclusion

Machine Translated by Google

With this module, Crack Chain gains evolutionary control without losing its protocol nature.

In other words:

• the system can grow, • the
system can be corrected, • the
system can migrate, • but it
cannot be deformed uncontrollably.

ÿ Next natural block

The next logical step is:

“Security, Threat Model & Adversarial Resistance”

This defines how the protocol resists attacks, manipulation, collusion, and node divergence.

Security, Threat Model & Adversarial Resistance

(Crack Chain Core Protocol—Defensive Integrity Layer)

---

1. Formal definition

The **Security, Threat Model & Adversarial Resistance** defines the set of assumptions, threats, controls, and defense mechanisms that protect Crack Chain against internal manipulation, external attacks, malicious nodes, and protocol divergence.

Its function is to ensure that the system remains:

• integrity, •
verifiable, •
deterministic, •
resistant to collusion, • and
operational under adverse conditions.

---

2. Guiding principle of security

Machine Translated by Google

System security does not depend on concealment, but on:

- reproducibility,
- consensus, •
traceability, • cross-
validation, • and automatic
rejection of invalid states.

Formally:

Security = Integrity + Determinism + Consensus + Auditability

---

3. Threat area

The threat model considers five main vectors:

3.1 Internal Threat

- malicious host node • corrupt auditor
- player attempting to alter
the flow of actions • operator attempting to introduce unauthorized
logic

3.2 External Threat

- Network attack •
Identity spoofing • Replay fraud • Invalid
message injection •
Deliberate desynchronization

3.3 Threat to consensus

- collusion between nodes • hash
forgery
- divergent state proposed as valid • unauthorized local branch
attempt

3.4 Threat of execution

Machine Translated by Google

• Deck alteration • Action log manipulation • Discrepancy between nodes on the same event • Final settlement corruption

3.5 Economic threat

• Fee abuse • Hand spam • Operational workload hoarding • Simulating participation to obtain undeserved rewards

---

4. Adversary Model

Crack Chain assumes that the adversary can be:

• **passive,** observing traffic and state, • **active,** attempting to modify messages or states, • **insider,** operating from an authorized node, • **coordinated,** acting as a group, • **economically opportunistic,** exploiting protocol incentives.

The system does not assume perfect honesty.
Assume **verifiable honesty.**

---

5. Integrity controls

5.1 Cryptographic signatures

All relevant actions must be signed.

```
Action = {

    payload,

    signature,

    signer_id

}
```

Machine Translated by Google

Any message without a valid signature will be rejected.

5.2 Hash chaining

Each state references the previous state.

State_t = H(State_t-1 + Action_t + Consensus_t)

This prevents:

- silent modification, • retroactive insertion, • unauthorized reordering, • event hiding.

5.3 Reproducible determinism

The state must be reconstructible by any valid node.
If the reconstruction does not match, the status is considered invalid.

5.4 Cross-validation

The nodes do not rely on a single report.

A comparison between multiple nodes is required to confirm:

- deck, • actions, • balances, • final payout, • state transition.

---

6. Attacks considered

Machine Translated by Google

6.1 Deck Manipulation

Attempt to alter the order or composition of letters.

Defense:

• PGDM v2 deterministic
• seed hash • deck
cross-validation
• rejection of any non-reproducible deck

6.2 Falsification of action

Attempt to send unauthorized fold/call/raise/check.

Defense:

• Digital signature
• Sequence control • Balance
validation • Turn ownership
verification

6.3 Replay attack

Reusing previous messages as if they were current.

Defense:

• session nonces • context
hash • required
previous_hash • duplicate rejection

6.4 State divergence

A node presents a state different from the consensus.

Defense:

• comparison of hashes

Machine Translated by Google

• majority validation •
isolation of the divergent node • rollback to
the last accepted state

6.5 Collusion attack

A group of nodes attempts to agree on a false state.

Defense:

• Qualified acceptance threshold • Load
distribution among nodes • Independent audit •
Anomalous correlation monitoring

6.6 Settlement manipulation

Attempt to alter the final distribution of the pot.

Defense:

• Payout derived from verified status • Settlement
linked to ledger • Double balance
validation
• closing with final hash

---

7. Resistance Mechanisms adversarial

7.1 Rejection due to inconsistency

If an event does not match the expected state, it is invalidated.

7.2 Node isolation

If a node repeatedly fails or introduces divergence, it is excluded from the active consensus.

Machine Translated by Google

7.3 Forensic Reproduction

Any hand can be reconstructed for full audit.

7.4 State Freeze

In the event of a critical anomaly, the system freezes the transition before compromising the ledger.

7.5 Controlled Rollback

If the error affects global consistency, it reverts to the last valid block.

---

8. Severity Classification

Severity = {

LOW: non-critical local inconsistency,

MEDIUM: partial divergence of nodes,

HIGH: attempted manipulation of the state,

CRITICAL: attack on consensus or settlement

}

Response by level:

• **LOW** ÿ registration and observation • **MEDIUM** ÿ revalidation • **HIGH** ÿ node isolation • **CRITICAL** ÿ rollback and reconfiguration of the active set

---

9. Tolerance rule

The system tolerates partial failures but does not tolerate ambiguity in the final state.

Partial_Failure = acceptable

Machine Translated by Google

Final_State_Ambiguity = unacceptable

This means that the network can survive faulty nodes, but cannot accept two valid results for the same hand.

---

10. Security monitoring

The protocol requires continuous monitoring of:

• latency between nodes •
hash divergence • error
frequency • anomalous statistical
behavior • concentration of validations among the
same nodes • suspicious patterns in player actions

---

11. Emergent security property

Crack Chain's security is not an external layer added to the system.
It is a structural consequence of its architecture.

In other words:

If the state cannot be verified, then it does not exist as a valid state.

That principle makes security a function of the protocol, not an operational embellishment.

---

12. Final definition of the module

Security_Threat_Model_Adversarial_Resistance = {

threat_surface,

adversary_model,

cryptographic_integrity,

Machine Translated by Google

```
    consensus_resistance,

    settlement_protection,

    rollback_and_isolation_rules

}
```

---

13. Technical Conclusion

This module completes the defensive part of the Core Protocol.

With it, Crack Chain is specified to withstand:

• state corruption, • deck manipulation, • stock forgery, • node collusion, • and settlement alteration.

The logic is simple and strict:

• what is verifiable remains, • what is inconsistent is rejected, • what is divergent is isolated, • what is not reproducible does not enter the system.

ÿ Next natural block

The next logical step is:

“Performance, Scalability & Load Distribution”

This defines how the network grows without losing determinism, how it distributes load among nodes, and how it maintains stability under more tables, more players, and more throughput.

Performance, Scalability & Load Distribution

(Crack Chain Core Protocol—Operational Scaling Layer)

Machine Translated by Google

1. Formal definition

The **Performance, Scalability & Load Distribution** section defines how Crack Chain maintains stable, deterministic, and verifiable execution while scaling:

• the number of tables, •
the number of players, • the
volume of actions, • the
number of active nodes, • and the
real-time consensus load.

Its function is not only to "make the system fast," but to ensure that growth does not destroy:

• state coherence, •
reproducibility, • security,
• nor the
operational economy of the network.

---

2. Guiding principle

The system must scale without losing determinism.

Scale_Up must preserve:

determinism

consensus_integrity

replayability

settlement_consistency

If growth introduces ambiguity, the system is considered poorly sized.

---

3. Load variables

The operational load of the network depends on a set of key variables:

Machine Translated by Google

3.1 Load per table

Each table generates load according to:

- Number of hands per hour • Number of active players • Decision frequency • Number of validations required

3.2 Load per node

Each node supports:

- Message processing • Event validation • State replay • Consensus participation • Ledger synchronization

3.3 Global load

The entire network is charged by:

- Simultaneous tables • Active game fans • Settlement congestion • Audit activity • Synchronization traffic between nodes

---

4. Functional load model

The total load is expressed as:

Load_total = f(Tables, Players, Actions, Validations, Audits, Settlements)

And the load per node as:

Load_node_i = f(Hands_i, Messages_i, Consensus_i, Audit_i)

Machine Translated by Google

5. Load distribution rule

The network should not depend on overloaded nodes.

The distribution should follow this principle:

Workload_i ÿ Capacity_i

Where capacity includes:

- Available CPU •
Operating latency •
Connection stability
- history of successful validation • prior
participation in consensus

---

6. Dynamic balancing

The system uses dynamic balancing to avoid bottlenecks.

6.1 Table layout

Tables are assigned to nodes in such a way that:

- no table depends on a single point of failure, • validation can be
replicated, • the state remains synchronizable.

6.2 Adaptive Redistribution

If a node exceeds the load threshold:

- Its allocation is reduced • Part of
the validation is transferred to neighboring nodes • The hash chain is
preserved • Continuity of the hand is
maintained

---

7. Saturation threshold

Machine Translated by Google

Each node has an operational limit.

Node_Saturation = Load_node_i / Capacity_node_i

Operational states:

• **Normal:** load within expected range • **High:** high but tolerable load • **Critical:** risk of delay or divergence • **Saturated:** the node stops receiving any new load

---

8. Horizontal scalability

Crack Chain scales by adding nodes, not by inflating a central node.

That means:

• More host nodes = more table capacity • More auditors = more parallel verification • More distribution = less central fragility

Architecture grows by **lateral expansion,** not by vertical concentration.

---

9. Limited vertical scalability

Internal improvement of each node is allowed, but with a structural limit.

It is permitted:

• Better hardware • Better bandwidth • More efficient validation • Better latency

Not allowed:

• Convert a node into a central authority

• Concentrate consensus on a single machine

Machine Translated by Google

- Replace the network with a master server

---

10. Operational Partition Model

The network can divide the work into logical partitions:

10.1 Partition by table

Each table operates as an independent state unit.

10.2 Partitioning by operating region

Nearby nodes can share validation pressure.

10.3 Partitioning by role

- Host nodes: execution • Auditor nodes: verification • Settlement nodes: shutdown • Reserve nodes: failover

---

11. Performance Metrics

The protocol must monitor, at a minimum, these metrics:

- consensus latency
- Letter distribution time • Action validation time • Settlement time • Retry rate

- Number of events per second • Number of simultaneous tables
- node divergence rate

---

12. Acceptable performance rule

Machine Translated by Google

The system is only functional if it maintains:

Response_Time_table + Consensus_Time + Settlement_Time <= Max_Operational_Window

If that window is exceeded:

- the workload is reduced,
- the work is redistributed, • or the new table assignment is postponed.

---

13. Relationship between performance and safety

Speed cannot break integrity.

The protocol prioritizes:

1. determinism
2. Consensus 3. Integrity 4. Performance

Never the other way around.

A fast but inconsistent network is invalid.
A slower but verifiable network is still valid.

---

14. Relationship with the economic model

The operational workload should be reflected in the incentive.

If a node supports more validation, more tables, or more traffic:

- Receives more proportionate reward • Justifies greater participation in fees • Maintains alignment between work and pay

This prevents the network from growing without the cost being matched by the compensation.

Machine Translated by Google

15. Emergent property of the system

Crack Chain does not scale like a conventional poker app.
Scale as a **verifiable distributed execution infrastructure.**

This implies:

- More activity does not break the system,
- the load is shared, •
consensus is maintained, •
settlement remains reproducible.

---

16. Final definition of the module

Performance_Scalability_Load_Distribution = {

workload_metrics,

dynamic_balancing,

saturation_thresholds,

horizontal_scaling,

limited_vertical_scaling,

partitioning_strategy,

settlement_performance_rules

}

---

17. Technical Conclusion

This module ensures that Crack Chain can grow while still remaining Crack Chain.

It's not just about "putting up with more traffic".
The aim is to keep the protocol's logic intact under real expansion.

Machine Translated by Google

If this module is met, the network can absorb more tables, more nodes, and more activity without losing:

- accuracy,
- protocol reliability,
- nor traceability.

ÿ Next natural block

The next logical step is:

“Interoperability, External Interfaces & API Surface”

It defines how Crack Chain connects with wallets, nodes, clients, identity systems, and external tools without compromising its deterministic core.

Interoperability, External Interfaces & API Surface

(Crack Chain Core Protocol—Integration Layer Specification)

---

1. Formal definition

The **Interoperability, External Interfaces & API Surface** section defines how Crack Chain exposes controlled capabilities to the outside without compromising its deterministic core.

Its goal is to allow:

- connection with game clients,
- interaction with wallets,
- communication between nodes,
- external auditing,
- integration with management tools,
- and eventual expansion to other compatible systems.

Every external interface must preserve:

- state integrity,
- protocol determinism,
- event traceability,
- and version control.

Machine Translated by Google

2. Guiding principle

Interoperability cannot introduce ambiguity.

External_Interface must preserve:

deterministic_core

state_integrity

message_order

version_compatibility

The core does not adapt to the interface.
The interface adapts to the core.

---

3. API surface area

The system's surface API does not expose sensitive logic without control.

3.1 Exposure is permitted

• node authentication
• table state • hand state

• action events • settlement results
• Verifiable history • Non-sensitive operational metrics

3.2 Exposure is not permitted

• private seeds • internal consensus paths • unsigned material • hidden validation rules • critical keys • internal logic not intended for public reproducibility

---

Machine Translated by Google

4. Access model

Every interface must be associated with an access level.

4.1 Public access

Allows limited reading of:

- visible status of tables, • availability,
- published results, •
events already completed.

4.2 Authenticated Access

Allows:

- participation in tables, • issuance
of signed shares, • consultation of private
session status, • interaction with wallet binding.

4.3 Node Access

Allows:

- validation,
- consensus,
- state playback, • host-to-host
communication, • hand block
broadcasting.

4.4 Audit access

Allows:

- Full replay, • Hash
verification, • Ledger comparison,
- Forensic analysis.

---

5. Interoperability Model

Machine Translated by Google

Crack Chain can interoperate at several levels:

5.1 Internal Interoperability

Between modules of the system itself:

- PGDM v2 •
HLCP •
LCCP Consensus • NHIO
Model
- settlement engine

5.2 Controlled external interoperability

With systems such as:

- wallets, •
client interfaces, •
dashboards, •
auditing tools, • identity services.

5.3 Future Interoperability

With auxiliary protocols provided that:

- Do not break determinism, • Do not
modify the core, • Do not alter the
historical ledger.

---

6. Types of interfaces

6.1 Client Interface

Interface for the player or end user.

Typical functions:

- Enter table • Sign
action • View visible
status
- Receive update

Machine Translated by Google

- Check final result

6.2 Host Interface

Interface for host nodes.

Typical functions:

- Receive events
- Validate transitions
- issue consensus • register status • publish hand block

6.3 Auditor Interface

Interface for audit nodes.

Typical functions:

- rebuild hands
- Verify hash chain • Compare logs • Generate evidence • Detect discrepancies

---

6.4 Settlement Interface

Settlement interface.

Typical functions:

- Execute payout • Close balances • Issue final confirmation
- Register hand closing

---

7. Minimum Surface API

Machine Translated by Google

The minimum API of the system must cover four families of functions.

7.1 Identity APIs

- Register node • Verify signature • Link wallet
- Validate role

7.2 Table APIs

- create table
- Join table
- Leave table • Check status

7.3 Hand APIs

- start hand
- Send action • Validate transition
- Check log

7.4 Settlement APIs

- Calculate payout
- close hand
- Issue final hash • Check closure

---

8. Interface design rule

Every API call must be:

- idempotent when applicable, • signed when it affects state, • versioned, • traceable,
- and validatable by an external node.

API_Call = {

Machine Translated by Google

```
    version,
    signer,
    timestamp_hash,
    payload,
    signature,
    replay_safe
}
```

---

9. External Message Protocol

External communication uses structured messages.

9.1 Minimum fields

• protocol_version •
message_type •
sender_id •
session_id •
hand_id •
payload •
signature •
previous_hash

9.2 Acceptance Rule

An external message is only valid if:

• belongs to the correct version, • is
signed, • respects
sequence, • matches
the session context, • does not break
local state.

Machine Translated by Google

10. Compatibility between external versions

The interface layer must support controlled transition.

Basic rule

A new interface can coexist with a previous one only if both produce:

• same logical state, • same effect on the ledger, • same reconstruction of the event.

If the format changes, the system must maintain an adapter or compatibility window.

---

11. Adapters and integration

The protocol allows adapters for:

• compatible wallets, • authentication systems, • monitoring tools, • alternative frontends, • reporting modules.

Every adapter should act as a translation layer, never as a decision layer.

---

12. Rules of non-interference

The outer layer cannot:

• alter deck rules, • alter the order of actions, • alter the consensus, • alter the settlement, • alter the hash chain.

The interface transmits, but does not govern.

Machine Translated by Google

---

13. Acceptable integration criterion

An integration is valid if it meets these conditions:

Valid_Integration =

state_preserved

ÿ signatures_verified

ÿ replay_consistent

ÿ compatible version

ÿ no_core_modification

---

14. Emergent property of the API surface

The Crack Chain API is not a gateway to system control.
It is a **controlled surface of verifiable interaction.**

This implies:

• the outside can read, •
can act within rules, • can audit, •
but cannot distort
the protocol.

---

15. Final definition of the module

Interoperability_External_Interfaces_API_Surface = {

access_model,

api_families,

Machine Translated by Google

```
    message_protocol,
    version_compatibility,
    adapter_rules,
    non_interference_constraints
}
```

---

16. Technical Conclusion

With this layer, Crack Chain can connect with the outside world without losing its protocol identity.

The operational conclusion is simple:

- the external is integrated,
- the internal remains stable, • the verifiable is preserved, • the incompatible is rejected.

ÿ Next natural block

The next logical step is:

“Data Model, Event Schema & Ledger Structure”

That defines exactly how hands, shares, nodes, balances, hashes, and blocks are represented within the system.

Data Model, Event Schema & Ledger Structure

(Crack Chain Core Protocol — Canonical Data Layer Specification)

Machine Translated by Google

1. Formal definition

The **Data Model, Event Schema & Ledger Structure** section defines how Crack Chain represents, organizes, and preserves all critical system information.

Its function is to establish a unique, consistent, and reproducible structure for:

- players, • nodes,
- tables,
- hands, •
actions, • states,
- settlements,
- and audit blocks.

The goal is for each piece of data to have a canonical and verifiable form within the protocol.

---

2. Guiding principle

All data that affects the system must be:

- structured, •
versioned, • signed
where appropriate, • traceable, • and
reproducible.

Valid_Data = structured + signed + versioned + traceable + replayable

---

3. Base data model

The system is composed of main protocol entities.

3.1 Player Entity

It represents a human or logical participant in the game.

Minimum fields:

Machine Translated by Google

```
Player = {
    player_id,
    wallet_id,
    role,
    status,
    balance,
    session_binding
}
```

3.2 Node Entity

It represents a node in the network.

```
Node = {
    node_id,
    node_type,
    capacity,
    load,
    reputation,
    consensus_status
}
```

Possible types:

- host node
- auditor node
- reserve node
- settlement node

Machine Translated by Google

3.3 Entity Table

It represents an active or historical table.

```
Table = {
    table_id,
    ruleset_id,
    host_nodes,
    players,
    current_hand_id,
    table_state
}
```

3.4 Hand Entity

It represents an individual hand.

```
Hand = {
    hand_id,
    table_id,
    seed_hash,
    deck_hash,
    action_log_hash,
    settlement_hash,
    final_status
}
```

Machine Translated by Google

4. Event Schema

Each event within the system must follow a homogeneous structure.

4.1 General definition

```
Event = {
    event_id,
    protocol_version,
    event_type,
    actor_id,
    table_id,
    hand_id,
    timestamp_hash,
    payload,
    signature,
    previous_event_hash
}
```

4.2 Mandatory properties

Every event must include:

• unique identifier, • event type, • emitting actor, • table context, • hand context, • reference to previous event, • valid signature, • and structured payload.

Machine Translated by Google

5. Types of events

5.1 Table Events

Table-related events.

Examples:

• table_created •
table_opened •
table_paused •
table_closed •
table_rebalanced

5.2 Hand Events

Events related to the hand.

Examples:

• hand_initialized •
deck_generated •
cards_dealt •
action_received •
action_validated •
showdown_executed •
pot_settled •
hand_finalized

5.3 Node Events

Events related to nodes.

Examples:

• node_registered •
node_synced •
node_validated •
node_penalized •
node_isolated •
node_reintegrated

Machine Translated by Google

5.4 Settlement Events

Liquidation-related events.

Examples:

- payout_calculated •
balance_updated •
settlement_committed •
settlement_rejected

---

6. Sequentiality Rule

Events cannot exist out of logical order.

Event_n must reference Event_n-1

This guarantees:

- correct chaining, • no duplication, •
no hidden insertion, •
no arbitrary reordering.

---

7. Ledger Structure

The system ledger is the canonical source of the historical state.

It is divided into three levels.

7.1 Hand Ledger

It contains the history of a specific hand.

Includes:

- seed,

Machine Translated by Google

• deck, •
actions, •
validations, • final
result, • closing hash.

7.2 Table Ledger

It contains the aggregated history of the table.

Includes:

• hands played, •
active nodes, • state
changes, • operational
behavior, • aggregated metrics.

7.3 Global Protocol Ledger

It contains the structural history of the system.

Includes:

• protocol versions, • upgrades,
• governance
events, • node penalties, •
global configuration changes.

---

8. Hash chain model

Each block of data must be linked to the previous one.

Ledger_Block_t = H(Ledger_Block_t-1 + Event_Set_t + Consensus_t)

This produces:

• historical continuity,

Machine Translated by Google

• cryptographic integrity, •
immediate tamper detection.

---

9. Canonical State

At all times there must be only one valid interpretation of the state.

Canonical_State = max(agreement among valid nodes)

If two nodes describe the same event differently:

• Verification is activated, • It is
compared against the ledger, • The
inconsistent version is rejected.

---

10. Data serialization

Serialization must be stable and predictable.

Requirements:

• fixed format, •
defined field order, • explicit types, •
cross-version
compatibility, • integrity validation before
parsing.

The system can use representations such as:

• Structured JSON, • binary
canonical encoding, • protobuf-like
schema, • or custom protocol
format.

The important thing is that serialization is **deterministic.**

Machine Translated by Google

11. Ledger reproducibility

Any valid node should be able to reconstruct the ledger from:

• signed events, •
previous hashes, •
protocol rules, • and
applied version.

Rebuilt_Ledger = Replay(Events, Ruleset, Version)

If the replay does not match the original ledger, the state is considered invalid.

---

12. Logical Immutability

Once a block enters the ledger:

• It is not edited,
• it is not overwritten, • it
is not reinterpreted without leaving a trace.

Corrections can only be submitted as:

• new events, •
protocol reversal, • or
compensatory blocks.

Never as a silent edition of the past.

---

13. Consistency validation

Each entry must pass validations of:

• type, •
sequence, •
signature,
• previous reference, •
state consistency, • version
compatibility.

Machine Translated by Google

Valid_Record =

type_ok

ÿ sequence_ok

ÿ signature_ok

ÿ previous_hash_ok

ÿ state_consistent

ÿ compatible version

---

14. Final definition of the module

Data_Model_Event_Schema_Ledger_Structure = {

canonical_entities,

event_schema,

sequential_integrity,

multi_layer_ledger,

hash_chain,

replayable_state

}

---

15. Technical Conclusion

With this module, Crack Chain obtains an internal data grammar.

That means the system stops relying on ambiguous interpretations and starts operating with:

- clear entities,

Machine Translated by Google

• verifiable events, •
reproducible ledger, •
and unique canonical state.

In practical terms: this block turns the protocol into a consistent information machine, not just a simple transactional application.

ÿ Next natural block

The next logical step is:

“Monitoring, Observability & Diagnostic Framework”

This defines how the system observes itself, how it reports anomalies, and how it produces operational evidence without violating privacy or determinism.

Monitoring, Observability & Diagnostic Framework

(Crack Chain Core Protocol—Runtime Inspection Layer)

---

1. Formal definition

The **Monitoring, Observability & Diagnostic Framework** section defines how Crack Chain observes its own operation in real time, detects anomalies, preserves operational evidence, and enables technical diagnostics without compromising protocol determinism.

Its goal is for the system to be able to answer these questions accurately:

• what happened,
• when it happened, •
at which node it happened,
• under which version it
happened, • what state it
produced, • and whether that state was valid or not.

---

2. Guiding principle

Machine Translated by Google

The system must be observable without becoming manipulable.

Observability must preserve:

protocol_integrity

state_determinism

audit_trail

privacy_constraints

Observability cannot alter performance.
You can only measure it, record it, and explain it.

---

3. Framework objectives

The framework fulfills five essential functions:

1. **Operational monitoring**

   • Measure load • Measure latency • Measure availability • Measure performance per node and per table 2. **Structural observability**

   • Track actions • Track transitions • Track settlement • Track consensus

3. **Diagnosis**

   • Identify faults • Isolate faulty nodes • Detect divergences • Reconstruct event chains

4. **Forensic evidence**

   • Preserve history • Demonstrate consistency • Support external audit 5. **Degradation prevention**

Machine Translated by Google

• anticipate saturation •
reduce risk of collapse • activate
operational redistribution

---

4. Domains of observation

4.1 Network Level

Measures:

• connectivity between nodes, •
synchronization latency, • message
loss, • consensus delays, •
channel availability.

4.2 Table level

Measures:

• table state, • activity per
hand, • player participation, •
action times, • settlement frequency.

4.3 Hand level

Measures:

• sequence of events, • deck
integrity, • validity of actions,
• consistency of transitions, •
correct closure.

4.4 Node level

Measures:

• load, •
reputation,

Machine Translated by Google

• errors,
• divergences, •
participation in consensus, • validation
history.

4.5 Protocol Level

Measures:

• version compatibility, • module
health, • upgrade stability,
• overall system behavior.

---

5. Observability Model

Observability is structured in three layers:

Observability = {

metrics,

logs,

traces

}

5.1 Metrics

Quantifiable values of the system.

Examples:

• hands per minute, •
average consensus time, • error rate
per node, • average settlement
latency, • percentage of load per host.

Machine Translated by Google

5.2 Logs

Structured event logs.

Examples:

• action received, • hash validated, • node penalized, • transition accepted, • hand completed.

5.3 Traces

Complete traces of a hand or session.

They allow reconstruction:

• exact chain of events, • execution order, • paths between nodes, • decision points, • causes of failure.

---

6. Observational record

All observability data must be:

• timestamped, • versioned, • context-bound, • immutable, • and replayable.

Observation = {

obs_id,

protocol_version,

scope,

source_id,

Machine Translated by Google

```
    context_id,
    value,
    timestamp_hash,
    signature
}
```

---

7. Observability without interference

Observability cannot:

• modify actions, • alter the deck, • influence consensus, • condition settlement, • or expose unauthorized sensitive material.

By design, the system observes from outside the decision-making plane.

---

8. Technical Diagnosis

The diagnosis is triggered when an observation exceeds a tolerable threshold.

8.1 Observable symptoms

• Unusual latency • Node desynchronization • Hash discrepancy • Settlement delay • Deck reconstruction divergence • Signature error • Availability drop

8.2 Diagnosis of the origin

The system attempts to determine if the anomaly originated in:

Machine Translated by Google

- network,
- node,
- table,
- hand lifecycle, •
consensus, •
settlement, • or
protocol version.

---

9. Alarm thresholds

Each metric has an operating threshold.

Threshold = {

warning,

degraded,

critical

}

9.1 Warning

The condition is still tolerable, but outside the expected range.

9.2 Degraded

The operation continues, but with a risk of accumulating errors.

9.3 Critical

The system must intervene immediately to prevent corruption or loss of consistency.

---

10. Detection process

The basic cycle is:

Machine Translated by Google

Observe ÿ Compare ÿ Classify ÿ Isolate ÿ Revalidate ÿ Report

Step by step:

- Observe data
- Compare them to the expected state • Classify the anomaly
- isolate the source • revalidate the state
- Report the diagnosis

---

11. Diagnostic isolation

If a node or table fails, the system must be able to isolate it without destroying the entire network.

11.1 Local isolation

It only affects the defective table or node.

11.2 Consensus isolation

Temporarily removes divergent nodes from the valid set.

11.3 Preventive isolation

It is activated before the fault becomes a global inconsistency.

---

12. Forensic evidence

The framework must preserve sufficient evidence to reconstruct any incident.

Includes:

- action logs, • hand hashes, • consensus validations, • final settlement,

Machine Translated by Google

- applied version,
- responsible node,
- sequence of events.

This evidence allows us to determine not only **what happened,** but **whether what happened was valid.**

---

13. Emergent property of observability

Crack Chain doesn't need to guess its status.

The system knows this, because:

- records,
- compares,
- validates,
- and preserves evidence.

This makes observability a structural function of the protocol, not a peripheral tool.

---

14. Final definition of the module

```
Monitoring_Observability_Diagnostic_Framework = {

    metrics_layer,

    logs_layer,

    traces_layer,

    anomaly_detection,

    diagnostic_classification,

    node_isolation,

    forensic_evidence

}
```

Machine Translated by Google

---

15. Technical Conclusion

This module completes the system's ability to see itself in real operation.

With it, Crack Chain can:

• detect degradation, •
explain failures, •
isolate divergences, •
and sustain a complete technical audit.

The central rule remains intact:

If the system cannot be observed accurately, then it cannot be governed safely.

ÿ Next natural block

The next logical step is:

“Deployment, Bootstrapping & Operational Lifecycle”

This defines how the network is deployed, how it starts from scratch, how nodes are incorporated, and how continuous operation is maintained.

Deployment, Bootstrapping & Operational Lifecycle

(Crack Chain Core Protocol — System Initialization and Runtime Continuity Layer)

---

1. Formal definition

The **Deployment, Bootstrapping & Operational Lifecycle** section defines how Crack Chain is deployed, how it starts from an empty state, how it incorporates active nodes, and how it maintains operational continuity throughout its lifecycle.

Its goal is to ensure that the system can transition from:

Machine Translated by Google

- **Zero active infrastructure** • A **validated functional network**
- without losing determinism, integrity or version control.

---

2. Guiding principle

The system startup must be reproducible and verifiable.

Deployment must preserve:

```
deterministic_initialization

version_control

consensus_readiness

auditability
```

There is no valid deployment if the initial state cannot be reconstructed.

---

3. Phases of the operating cycle

The operational life cycle of the system is divided into six phases:

```
Lifecycle = {

    pre_deployment,

    bootstrap,

    activation,

    steady_state,

    maintenance,

    retirement

}
```

Machine Translated by Google

3.1 Pre-Deployment

Phase prior to the actual activation of the protocol.

Includes:

- version verification,
- configuration validation,
- candidate node verification,
- consensus rule review,
- interface compatibility check.

Productive traffic is not yet accepted at this stage.

3.2 Bootstrap

Structural start-up phase of the network.

Includes:

- generation of the initial state,
- registration of founder nodes,
- synchronization of global parameters,
- creation of the base ledger,
- activation of the first set of rules.

The bootstrap must produce a unique and reproducible initial state.

3.3 Activation

Phase in which the network is enabled to operate.

Includes:

- opening of tables,
- enrollment of players,
- enabling of the HLCP cycle,
- issuance of initial seeds,
- validation of the first consensus.

Machine Translated by Google

3.4 Steady State

Stable operating phase.

Includes:

• active hands, • continuous consensus, • recurring settlement, • node monitoring, • load distribution, • permanent audit.

3.5 Maintenance

Controlled maintenance phase.

Includes:

• upgrades, • node rotation, • patches, • performance optimization, • status fixes, • post-change validations.

3.6 Retirement

Closing phase of an instance or version.

Includes:

• Freezing new hands, • Completion of pending states, • Ledger export, • Session closure, • Historical preservation.

---

4. Bootstrap model

Machine Translated by Google

The system bootstrap is not an arbitrary start.
It is a controlled construction of canonical state.

4.1 Minimum Inputs

• Protocol version • Initial set of nodes • Rule parameters • Consensus configuration • Settlement parameters • Security configuration • Base data schema

4.2 Minimum output

```
Bootstrap_Output = {

    genesis_state,

    protocol_version,

    node_registry,

    consensus_parameters,

    ledger_root_hash

}
```

---

5. Genesis state

The **genesis state** is the foundational state of the system.

It must contain:

• protocol identity, • base version, • initial rules, • founder node configuration, • ledger root hash.

It cannot be generated differently in two equivalent deployments without breaking reproducibility.

Machine Translated by Google

---

6. Node Registration

Before activation, each node must be registered.

6.1 Minimum node fields

```
Node_Record = {
    node_id,
    node_type,
    role,
    capacity,
    signature_key,
    software_version,
    status
}
```

6.2 Node States

- pending
- verified
- active
- degraded
- isolated
- retired

---

7. Boot sequence

The formal start follows this sequence:

Initialize ÿ Verify ÿ Register ÿ Sync ÿ Commit ÿ Activate

Machine Translated by Google

7.1 Initialize

Rules, parameters, and base structure are loaded.

7.2 Verify

The binary, version, and configuration are validated to ensure compatibility.

7.3 Register

Permitted nodes are added to the active registry.

7.4 Sync

The seed, ledger root, and global parameters are synchronized.

7.5 Commit

The initial state is confirmed as the basis of operation.

7.6 Activate

The network is now available for productive hands and tables.

---

8. Deployment Model

The system can be deployed in different modes, as long as the core protocol is not broken.

8.1 Local Deployment

Used for testing, simulation, and internal validation.

8.2 Federated Deployment

Used when multiple nodes cooperate under a common configuration.

8.3 Distributed Deployment

Used in real operational network with multiple host nodes and auditors.

Machine Translated by Google

8.4 Staged Deployment

Used for partial migrations and gradual version activation.

---

9. Operational lifecycle

Once activated, the system enters continuous operational life.

9.1 Operative loop

Hand_Execution ÿ Consensus_Check ÿ Settlement ÿ Logging ÿ Monitoring ÿ
Next_Hand

9.2 Cycle Property

Each cycle must:

• maintain consistent state, • preserve
history, • produce verifiable
hash, • be recoverable by replay.

---

10. Maintenance lifecycle

Maintenance cannot alter the system's history.

10.1 Allowed maintenance

• node upgrade, • performance
tuning, • bug fixes, • key rotation,
• load rebalancing.

10.2 Forbidden maintenance

Machine Translated by Google

• Silent ledger rewrite, • Unversioned rule change, • Hidden deck alteration, • Retroactive settlement modification.

---

11. Node lifecycle

Each node follows its own cycle within the system.

```
Node_Lifecycle = {
    onboarded,
    verified,
    active,
    monitored,
    degraded,
    isolated,
    reintegrated_or_retired
}
```

11.1 Onboarded

The node is proposed for entry.

11.2 Verified

The node passes technical and identity checks.

11.3 Active

The node participates in consensus and execution.

11.4 Monitored

The node is continuously observed.

11.5 Degraded

Machine Translated by Google

The node presents an operational risk.

11.6 Isolated

The node is temporarily removed from the consensus.

11.7 Reintegrated or retired

The node resumes operation or is decommissioned in a controlled manner.

---

12. Recovery model

If the system is interrupted, it must be able to return to a coherent state.

12.1 Recovery from snapshot

It is restored from a previously validated state.

12.2 Recovery from ledger

The state is reconstructed by replaying signed events.

12.3 Recovery from consensus

The latest state of qualified consensus is taken as truth.

---

13. Business continuity rules

Continuous operation requires that:

• there is no ambiguity in the hand state, • there is no double settlement, • there is no unauthorized fork, • there is no active node without a compatible version, • there is no table without logical backup.

Operational_Continuity =

valid_state

Machine Translated by Google

ÿ valid_consensus

ÿ valid_settlement

ÿ valid_versioning

ÿ valid_node_set

---

14. Shutdown protocol

The system shutdown must be orderly.

14.1 Controlled shutdown

• Freeze new hands • End active hand
• close table
• Export ledger •
Preserve logs • Mark nodes on output

14.2 Emergency shutdown

It is only used in cases of critical corruption.

• Immediate freezing • Blocking
of new actions • Preservation of evidence
• Rollback to the last valid block

---

15. Emergent property of the operating cycle

Crack Chain doesn't "turn on" like an app.

It is **instantiated,** verified , activated , observed , and **maintained** as a protocol infrastructure.

Machine Translated by Google

That means the system lifecycle is part of the protocol, not an external task.

---

16. Final definition of the module

```
Deployment_Bootstrapping_Operational_Lifecycle = {

    pre_deployment_checks,

    genesis_bootstrap,

    node_registration,

    activation_sequence,

    steady_state_operation,

    maintenance_rules,

    recovery_paths,

    shutdown_protocol

}
```

---

17. Technical Conclusion

With this block, Crack Chain obtains a complete definition of start, operation, and closure.

The network no longer depends solely on rules of the game or consensus.
It also has formalized:

- how it is born,
- how it is maintained,
- how it recovers,
- and how it is extinguished without losing integrity.

ÿ Next natural block

Machine Translated by Google

The next logical step is:

“Appendix: Formal Definitions, Notation & Terminology”

This is where all the definitions, abbreviations, symbols and terms used in the protocol are consolidated to make the document completely normative.

Appendix: Formal Definitions, Notation & Terminology

(Crack Chain Core Protocol — Canonical Specification Layer)

---

1. Definition of the appendix

This appendix establishes the **formal glossary, mathematical notation, canonical definitions, and operational terminology** used throughout the Crack Chain protocol.

Its function is to eliminate semantic ambiguity and ensure that any implementation interprets the system identically.

---

2. Principle of interpretation

All terms MUST be interpreted:

context-free

version-bound

deterministically

schema-consistent

If a definition conflicts with execution, the structural specification of the protocol prevails over natural language.

---

3. General notation

Machine Translated by Google

3.1 Structures

• {} ÿ structured object • [] ÿ ordered
list • () ÿ logical grouping ÿ ÿ
transformation or result
•

• **ÿ** ÿ logical conjunction (AND) •
ÿ ÿ logical disjunction (OR)

3.2 Functions

• H(x) ÿ cryptographic hash function of x •
f(x) ÿ deterministic function of the system •
Consensus(x) ÿ result of distributed validation •
Transition(S, A) ÿ change of state S under action A • Replay(L)
ÿ reconstruction of state from ledger L

3.3 Identifiers

• id ÿ unique entity identifier
• hash ÿ cryptographic representation of state •
signature ÿ cryptographic proof of authorship •
timestamp_hash ÿ deterministic timestamp

---

4. Canonical definitions of the system

4.1 State

State_t = representation of system at time t

Includes:

• players

Machine Translated by Google

- nodes
- table
- actions • deck
- partial results

4.2 Event

Event = atomic unit of state transition

An event is immutable and chained to the previous one by means of a hash.

4.3 Node

Node = computational participant in consensus network

Could be:

- host •
auditor •
settlement
- reserve

4.4 Hand

Hand = complete execution cycle of a poker game instance

Includes:

- seed generation • deck
creation
- action loop •
consensus validation
- settlement

Machine Translated by Google

4.5 Ledger

Ledger = ordered sequence of immutable events

Properties:

- append-only • hash-chained • replayable
- verifiable

4.6 Seed

Seed = deterministic entropy source derived from system state

It is not external random.
It depends on the state of the system.

4.7 Deck

Deck = deterministic permutation of base card set

Generated exclusively by PGDM v2.

4.8 Consensus

Consensus = majority validation of state among active nodes

Basic rule:

Valid if valid_votes ÿ 2/3 active_nodes

Machine Translated by Google

4.9 Settlement

Settlement = final deterministic distribution of pot based on validated state

It cannot be altered after the final hash.

---

5. Identity Rules

5.1 Uniqueness

Each entity must have a globally unique identifier.

ÿ entity ÿ! id

5.2 Immutability

Once issued:

- event
- state hash •

ledger block

It cannot be modified without generating a new event.

5.3 Reproducibility

Replay(system_state) must equal original_state

Machine Translated by Google

6. Critical system functions

Function	Definition
H(x)	cryptographic hash
Consensus(x)	distributed validation
Transition(S,A)	state evolution
PGDM_v2()	deterministic deck generation
HLCP()	complete hand cycle
NHIO()	economic model of nodes

---

7. Types of state

7.1 Initial State

System or table startup state.

7.2 Intermediate State

State during execution of the hand.

7.3 Final State

Status closed after settlement.

Machine Translated by Google

7.4 Invalid State

State rejected due to inconsistency or lack of consensus.

---

8. Types of validation

8.1 Structural validation

Check data format.

8.2 Cryptographic validation

Verify signatures and hashes.

8.3 Consensus validation

Verify agreement between nodes.

8.4 Deterministic validation

Verify reproducibility of the state.

---

9. Execution Semantics

The system always operates under this model:

System = f(initial_state, events, consensus_rules)

There is no valid non-deterministic execution.

---

10. System truth hierarchy

If there is a conflict between interpretations:

Machine Translated by Google

1. Ledger hash chain (highest priority)
2. State reproduced by consensus 3.
Protocol rules (CCCP)
4. Definition of the specific module
5. Textual description

---

11. Critical Terminology

Deterministic

Completely predictable result from inputs.

Immutable

Not modifiable without breaking the integrity of the system.

Verifiable

It can be verified by any valid external node.

Reproducible

It can be rebuilt from scratch.

Consensus-bound

Dependent on distributed validation.

---

12. Rule of non-ambiguity

If the term is ambiguous:

use protocol definition

else:

reject interpretation

Machine Translated by Google

---

13. Conclusion of the appendix

This appendix consolidates Crack Chain as a fully formalizable system:

- without semantic ambiguity •
without free interpretation of the state
- with strict deterministic rules • with
a universally reproducible structure

---

14. Closing the complete document

With this block, the **Crack Chain Core Protocol** is fully defined as:

a deterministic distributed computer system for execution, validation and settlement of poker games under verifiable consensus.